

รายงานโครงการ: ระบบจำลองเส้นทางอพยพ ฉุกเฉินในอาคารด้วยทฤษฎีกราฟและการหาค่า เหมาะสมที่สุด (ฉบับสมบูรณ์ที่สุด)

1. นิยามระบบอย่างเป็นทางการ (Formal System Definition)

เราจำลองอาคารเป็นกราฟถ่วงน้ำหนักเชิงทิศทาง $G = (V, E, w)$ เพื่อใช้ในการวิเคราะห์เส้นทางอพยพที่เหมาะสมที่สุด

องค์ประกอบของกราฟ

- V (Vertices): เซตของจุดยอด $|V| = 21$ แทนตำแหน่งทางกายภาพในอาคาร (ห้อง, โถงทางเดิน, บันได, ทางออก)
- E (Edges): เซตของเส้นเชื่อม แทนเส้นทางเดินที่เชื่อมต่อระหว่างจุด โดยมีทิศทางกำกับเพื่อจัดระเบียบการไหล (Flow Control)
- $w(e)$ (Weight Function): ฟังก์ชันที่กำหนด "ต้นทุน" ของการเดินทางผ่านเส้นเชื่อม $e \in E$

สูตรคำนวณน้ำหนัก $w(e)$ และการวิเคราะห์ตัวแปร

$$w(e) = \frac{d(e)}{v_{base} \cdot (1 - \rho(e)) \cdot (1 - R(e))}$$

คำอธิบายตัวแปร:

- $d(e)$: ระยะทางจริง (เมตร) - เป็นค่าคงที่ตามโครงสร้างอาคาร
- v_{base} : ความเร็วพื้นฐาน (เมตร/วินาที) - โดยทั่วไปคือ 1.2 - 1.5 m/s สำหรับมนุษย์ปกติ
- $\rho(e)$: ความหนาแน่นของผู้ชน ($0 \leq \rho < 1$) - ยิ่งคนเยอะ ความเร็วจะลดลงตามทฤษฎีการไหลของของไหล (Fluid Dynamics)
- $R(e)$: ระดับความเสี่ยง ($0 \leq R < 1$) - แทนค่าความเข้มข้นของควันไฟหรือความร้อนที่บั่นทอนความสามารถในการเคลื่อนที่

2. การพิสูจน์ความเหมาะสมที่สุดของ Dijkstra (Proof by Contradiction)

ทฤษฎีบท (Theorem): สำหรับกราฟ $G = (V, E)$ ที่มี $w(e) \geq 0$ สำหรับทุก $e \in E$ อัลกอริทึมของ Dijkstra จะหาเส้นทางที่สั้นที่สุดจากโหนดเริ่มต้น s ไปยังโหนดใดๆ v ได้อย่างถูกต้องเสมอ

บทพิสูจน์โดยการหาข้อขัดแย้ง (Proof by Contradiction):

- สมมติให้ u เป็นโหนดแรกที่ Dijkstra เลือกดึงออกจาก Priority Queue แล้วกำหนดระยะทาง $d(u)$ แต่ค่านี้ **ไม่ใช่** ระยะทางที่สั้นที่สุดจริง (นั่นคือ มีเส้นทางอื่น P' ที่สั้นกว่า)
- ให้ P' เป็นเส้นทางที่สั้นที่สุดจริงจาก s ไปยัง u เนื่องจาก s อยู่ในเซตที่ประมวลผลแล้ว (S) และ u ยังไม่อยู่ใน S ในตอนแรก ดังนั้นต้องมีเส้นเชื่อม (x, y) ในเส้นทาง P' ที่ $x \in S$ และ $y \notin S$
- จากสมบัติของ Dijkstra ณ ขณะนั้น $d(y) = d(x) + w(x, y)$ และเนื่องจาก y ปรากฏก่อน u ในเส้นทาง P' และค่าน้ำหนักทุกเส้นเชื่อมไม่เป็นลบ ($w(e) \geq 0$) จะได้ว่า: $dist_{true}(u) \geq dist_{true}(y)$
- แต่เนื่องจาก Dijkstra เลือก u ก่อน y หมายความว่า $d(u) \leq d(y)$
- เมื่อรวมสมการจะได้ $d(u) \leq d(y) \leq dist_{true}(u)$ ซึ่งหมายความว่า $d(u)$ ต้องเป็นระยะทางที่สั้นที่สุดจริง ขัดแย้งกับข้อสมมติในข้อ 1
- สรุป:** เนื่องจาก $w(e)$ ในระบบของเราเป็นบวกเสมอ ($d(e) > 0$ และปัจจัยลบไม่เกิน 1) Dijkstra จึงให้คำตอบที่เป็น **Global Optimum** เสมอ

3. Connectivity — เงื่อนไขความปลอดภัยเชิงโครงสร้าง

คำถาม: “ถ้าปิดเส้นทางไป k เส้น อาคารต้องมีทางออกอย่างน้อยกี่จุด ถึงจะรับประกันได้ว่าทุกห้องยังมีเส้นทางออกอยู่”

บทพิสูจน์โดยใช้ Menger's Theorem: ทฤษฎีของ Menger (Edge Version) ระบุว่า: “จำนวนเส้นเชื่อมที่น้อยที่สุดที่ต้องตัดออกเพื่อทำให้โหนด u และ v แยกจากกัน เท่ากับจำนวนเส้นทางที่ไม่มีเส้นเชื่อมร่วมกัน (Edge-disjoint paths) ที่มากที่สุดระหว่าง u และ v ”

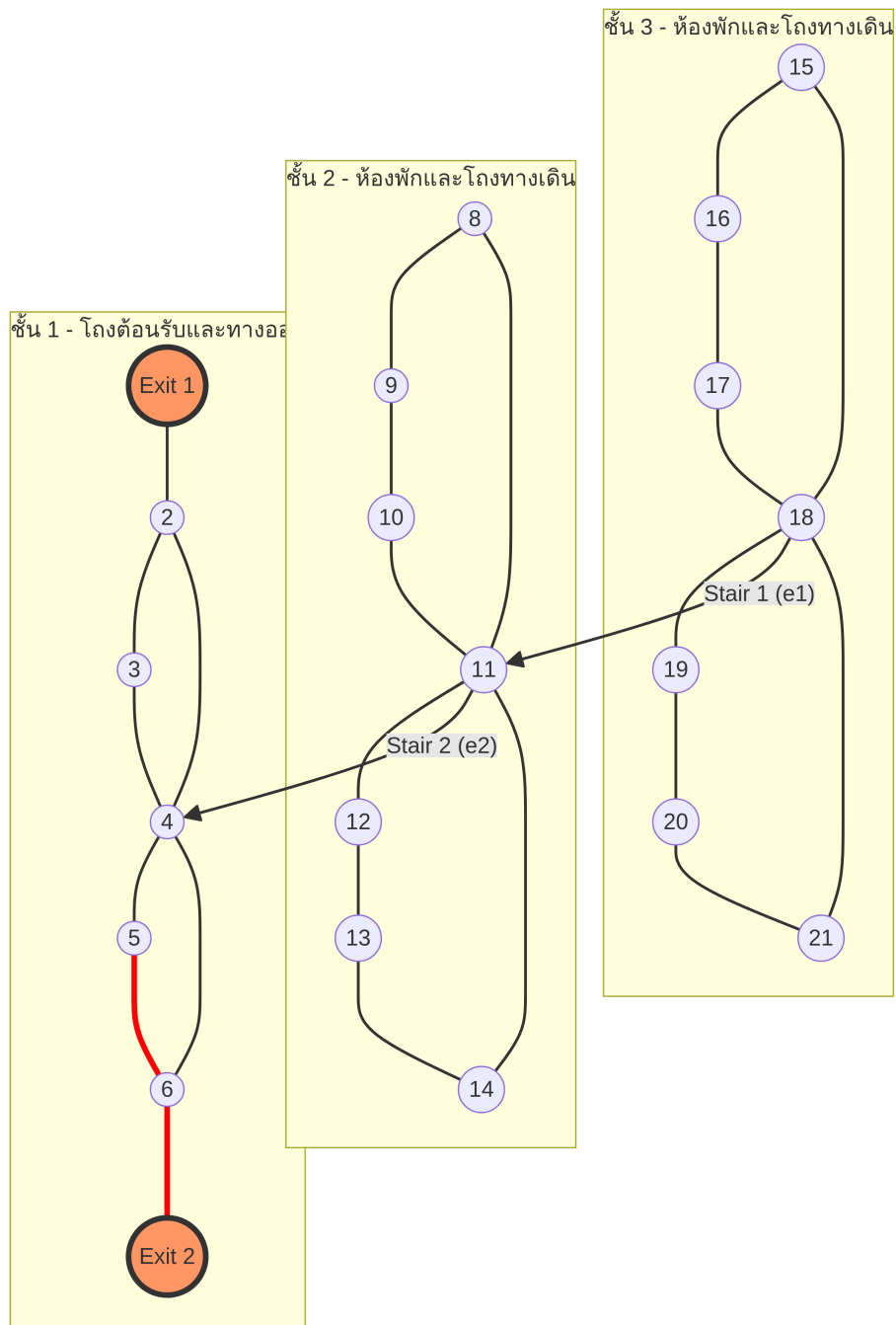
- เพื่อให้ทุกห้องยังสามารถเข้าถึงทางออกได้แม้จะสูญเสียเส้นเชื่อมไป k เส้น กราฟต้องมีความเชื่อมโยงของเส้นเชื่อม (Edge Connectivity) $\lambda(G) \geq k + 1$

2. ตามทฤษฎี Menger หมายความว่าต้องมีเส้นทางที่แยกจากกันโดยสิ้นเชิงอย่างน้อย $k + 1$ เส้นทางจากทุกห้องไปยังเซตของทางออก
 3. **สรุปเชิงโครงสร้าง:** เพื่อรับประกันความปลอดภัยภายใต้ความเสียหาย k จุด อาคารต้องมีทางออกที่เข้าถึงได้ผ่านเส้นทางที่แยกจากกันอย่างน้อย $k + 1$ จุด เพื่อให้แม้จะเสียไป k เส้นทาง ก็ยังเหลืออย่างน้อย 1 เส้นทางที่ใช้งานได้เสมอ
-

4. Bottleneck — การวิเคราะห์จุดอ่อนของอาคาร

Min-cut Max-flow Theorem: ปริมาณการไหลสูงสุดจาก Source ไปยัง Sink เท่ากับค่าความสามารถรวมต่ำที่สุดของ Cut ที่แยก Source ออกจาก Sink

การวิเคราะห์อาคารตัวอย่าง (21 โหนด)



การกำหนดค่า Capacity (c):

- ทางเดินภายในชั้น (e_{hall}): $c = 10$ คน/นาที
- บันไดเชื่อมระหว่างชั้น (e_{stair}): $c = 15$ คน/นาที

การเปรียบเทียบค่า Cut:

1. **Cut A (Exit Cut):** ตัดเส้นเชื่อมรอบทางออก 1 และ 7 (รวมเส้นเชื่อมภายในชั้นที่มุ่งสู่ทางออก)

- มีเส้นทางเข้าสู่ทางออกรวม 4 เส้น (จากโหนด 2, 6 และเส้นเชื่อมภายในชั้น)
- $Capacity(CutA) = 10 \times 4 = 40$ คน/นาที

2. **Cut B (Stair 2 Cut):** ตัดเส้นเชื่อม $e_2 = (11, 4)$

- เป็นเส้นทางหลักเส้นเดียวที่เชื่อมชั้น 2-3 ลงสู่ชั้น 1
- $Capacity(CutB) = 15$ คน/นาที

3. **Cut C (Stair 1 Cut):** ตัดเส้นเชื่อม $e_1 = (18, 11)$

- เป็นเส้นทางหลักเส้นเดียวที่เชื่อมชั้น 3 ลงสู่ชั้น 2
- $Capacity(CutC) = 15$ คน/นาที

วิเคราะห์ผล: เนื่องจาก $Capacity(CutB) = Capacity(CutC) = 15$ ซึ่งน้อยที่สุดในระบบ ดังนั้น **Bottleneck** คือบันไดหลัก จุดนี้จำกัดอัตราการไหลของทั้งอาคารให้ไม่เกิน 15 คน/นาที

5. สูตร Safety Score และการทดสอบ

เรากำหนดสูตรคะแนนความปลอดภัย (S) ที่สะท้อนความเป็นจริงเชิงวิศวกรรม ดังนี้:

$$S = 40 \cdot \min\left(1, \frac{N_{exit}}{V_{total}/5}\right) + 30 \cdot \min\left(1, \frac{C_{min-cut}}{P_{total}/10}\right) + 30 \cdot \left(\frac{\lambda(G)}{k+1}\right)$$

เหตุผลและการให้น้ำหนัก:

- Exit Score (40%):** วัดจำนวนทางออกเทียบกับขนาดอาคาร โดยมีเกณฑ์อ้างอิงว่าอาคารที่ปลอดภัยควรมีทางออกอย่างน้อย 1 จุดต่อทุก 5 โหนด (หรือพื้นที่เทียบเท่า) หากมีตามเกณฑ์นี้จะได้คะแนนเต็ม 40
- Bottleneck Capacity (30%):** วัดว่าจุดคอขวดของอาคารสามารถรองรับการอพยพได้ดีเพียงใดเมื่อเทียบกับจำนวนคนทั้งหมด (P_{total}) หาก Min-cut สามารถรองรับคนได้ตามสัดส่วนที่เหมาะสม จะได้คะแนนเต็ม 30
- Connectivity (30%):** วัดความทนทานของอาคารต่อการสูญเสียเส้นทาง (เช่น ไฟไหม้ปิดทาง) หากอาคารมีความเชื่อมโยงสูง จะได้คะแนนเต็ม 30

การทดสอบกับอาคารตัวอย่าง (Case Study)

ข้อมูลตัวอย่าง:

- $V_{total} = 21$ โหนด, $N_{exit} = 2$ จุด

- $C_{min-cut} = 15$ คน/นาที, $P_{total} = 100$ คน
- $\lambda(G) = 1$ (มีจุดตัดเดียวที่บันได), $k = 1$ (เป้าหมายรองรับความเสียหาย 1 จุด)

แทนค่าคำนวณ:

1. **Exit Score:** $40 \cdot \min\left(1, \frac{2}{21/5}\right) = 40 \cdot \min\left(1, \frac{2}{4.2}\right) = 40 \cdot 0.476 \approx 19.05$
2. **Bottleneck Score:** $30 \cdot \min(1, 15/(100/10)) = 30 \cdot \min(1, 1.5) = 30 \cdot 1 = 30$
3. **Connectivity Score:** $30 \cdot (1/(1+1)) = 15$

คะแนนรวม $S = 19.05 + 30 + 15 = 64.05/100$

วิเคราะห์ผล: คะแนน 64.05 สะท้อนว่าอาคารนี้ “พอใช้ได้ แต่ต้องปรับปรุง” แม้ Bottleneck จะพอรับมือได้ แต่คะแนน Exit Score ที่ต่ำ (19.05 จาก 40) และ Connectivity Score ที่ต่ำ (15 จาก 30) ชี้ให้เห็นว่าอาคารยังขาดทางออกที่เพียงพอและมีความเปราะบางต่อการสูญเสียเส้นทาง ควรพิจารณาเพิ่มทางออกหรือเส้นทางสำรองเพื่อเพิ่มความปลอดภัย

6. วิเคราะห์ Complexity

Algorithm	Time Complexity	ความเหมาะสมเชิงประยุกต์
Dijkstra	$O((V + E) \log V)$	เหมาะกับอาคารที่ซับซ้อนมากและไม่มีข้อมูลพิกัด GPS ภายในอาคาร
A*	$O((V + E) \log V)$	เหมาะกับอาคารที่มีแผนผังชัดเจน โดยใช้ระยะทาง Euclidean เป็น Heuristic

ทำไม A* ถึงมีประสิทธิภาพสูงกว่าในทางปฏิบัติ? แม้ Worst Case จะเท่ากัน แต่ A* ใช้ฟังก์ชัน $f(n) = g(n) + h(n)$ เพื่อเลือกสำรวจโหนดในทิศทางที่มุ่งหน้าสู่ทางออกก่อนเสมอ ทำให้ลดจำนวนโหนดที่ต้องประมวลผล (Search Space) ลงอย่างมากเมื่อเทียบกับ Dijkstra ที่กระจายการค้นหาออกทุกทิศทางแบบวงกลม